

Available online at www.sciencedirect.com

ScienceDirect

journal homepage: www.elsevier.com/locate/coseComputers
&
Security

Neural network laundering: Removing black-box backdoor watermarks from deep neural networks

William Aiken^a, Hyounghick Kim^{b,*}, Simon Woo^b, Jungwoo Ryoo^c^aDepartment of Electrical and Computer Engineering, College of Information and Communication Engineering, Sungkyunkwan University (SKKU), 16419, South Korea^bDepartment of Software, College of Software, Sungkyunkwan University (SKKU), 16419, South Korea^cProfessor of Information Sciences and Technology (IST) and the head of the Division of Business, Engineering, and IST, The Pennsylvania State University, Altoona, PA 16601, USA

ARTICLE INFO

Article history:

Received 23 May 2020

Revised 8 March 2021

Accepted 1 April 2021

Available online 18 April 2021

Keywords:

Neural networks

Intellectual property

Machine learning

Watermarking

Backdoors

ABSTRACT

Creating a state-of-the-art deep-learning system requires vast amounts of data, expertise, and hardware, yet research into copyright protection for neural networks has been limited. One of the main methods for achieving such protection involves relying on the susceptibility of neural networks to backdoor attacks in order to inject a watermark into the network, but the robustness of these tactics has been primarily evaluated against pruning, fine-tuning, and model inversion attacks. In this work, we propose an offensive neural network “laundering” algorithm to remove these backdoor watermarks from neural networks even when the adversary has no prior knowledge of the structure of the watermark. We can effectively remove watermarks used for recent defense or copyright protection mechanisms while retaining test accuracies on the target task above 97% and 80% for both MNIST and CIFAR-10, respectively. For all watermarking methods addressed in this paper, we find that the robustness of the watermark is significantly weaker than the original claims. We also demonstrate the feasibility of our algorithm in more complex tasks as well as in more realistic scenarios where the adversary can carry out efficient laundering attacks using less than 1% of the original training set size, demonstrating that existing watermark-embedding procedures are not sufficient to reach their claims.

© 2021 Elsevier Ltd. All rights reserved.

1. Introduction

Deep neural networks (DNNs) have become the state-of-the-art standard across a wide variety of fields ranging from computer vision to speech recognition systems, and they have been adopted by many industries (Abiodun et al., 2018). As a result, some organizations have come to rely heavily on

neural networks as part of their core operations, which frequently requires a substantial investment in powerful computing resources, vast quantities of data, and specialized machine learning expertise. Hence, organizations that do build and train their own models would need to protect their systems from plagiarism, and those who sell or share their models would also want to demonstrate ownership of their system should an infringement of copyright occur.

* Corresponding author.

E-mail addresses: billzo@skku.edu (W. Aiken), hyoung@skku.edu (H. Kim), swoo@g.skku.edu (S. Woo), jryoo@psu.edu (J. Ryoo).
<https://doi.org/10.1016/j.cose.2021.102277>

0167-4048/© 2021 Elsevier Ltd. All rights reserved.

Thus far, attempts to develop provable ownership in neural networks have mainly relied on two distinct categories of watermarking techniques: (1) Watermarks that are embedded through backdooring attacks by injecting the backdoor via training images (Adi et al., 2018; Zhang et al., 2018); and (2) Watermarks created and embedded directly into the structure of the neural network (Rouhani et al., 2019; Uchida et al., 2017). Our work focuses on the first set of watermarking techniques, which we refer to as “black-box” watermarks. Our work does not focus on “white-box” watermarks nor on techniques like (Rouhani et al., 2019) that deliberately focus on watermarking methods that preserve “the intrinsic distribution of weights/activations”.

Within this paper, we utilize the term “black-box” to describe the watermarks because the potential victim is unable to access the weights, loss, or logits from the neural network suspected to have been stolen but instead attempts to prove ownership by using only the final classification output via, for example, an API. Moreover, we utilize the term “backdoor” to focus on a specific subset of watermarks whereby the owner wishes to surreptitiously exploit the overparameterization of the network to output an unexpected result given very specific input. In these types of watermarks, the technique typically involves embedding specially-crafted inputs into the training of a neural network designed to have a highly consistent, but unusual, output during testing. For example, a watermark may be embedded into a network by including a subset of images during initial training that can skew the network to classify those images unexpectedly. This subset may involve a “trigger set” of unrelated images (Adi et al., 2018), or it may contain content or noise overlaid on the image (Zhang et al., 2018).

Feeding these inputs into the trained system returns a consistent output that would not be expected normally. The owner organization can then use the network’s output to demonstrate ownership of the model because only that specific watermarked model would behave in this way. That is, backdoor watermarks attempt to prove ownership of a model using only public API access by querying the potentially plagiarized network with carefully constructed inputs. Similarly, traditional backdoors on neural networks also exploit this overparameterization of neural networks; however, while a backdoor is generally used by adversaries for malicious ends (e.g., misclassifying stop signs with stickers as speed limit signs Gu et al., 2017), backdoor watermarks are used against adversaries to deter deployment of stolen models. In short, our work is an *offensive approach* against watermarks because our technique aims to remove a neural network’s watermark, regardless of who inserts it or how.

In this work, we present our novel neural network “laundering” algorithm to remove potentially watermarked neurons or channels in DNN layers. We achieve this via a three-step process of watermark recovery, detecting and resetting watermarked neurons, and retraining on the reconstructed watermarks and watermark masks, each step taking advantage of novel contributions. To show the effectiveness of our laundering algorithm against backdoor watermarks and to highlight their exaggerated claims of robustness, we compare the results of our method with the removal attempts made within the original watermark proposal papers (Adi et al., 2018; Zhang et al., 2018).

Our approach shows weaknesses of current deep neural network watermarking techniques – we can defeat the state-of-the-art watermarking techniques proposed by Zhang et al. (2018) and Adi et al. (2018). In particular, we show that with an appropriate representation of the variety of training data, adversaries who have no knowledge of the watermark can successfully remove the majority of watermarking techniques, and more importantly, our approach retains accuracy significantly higher than stated in the original work, (e.g., up to 20% higher in some cases Zhang et al., 2018).

We make the following contributions in this paper:

- We present our novel neural network “laundering” algorithm, which effectively removes neurons or channels in DNN layers that contribute to the classification of watermarked images. We evaluate our effectiveness under various limited training sets to which an attacker may have access.
- We highlight the previously-overlooked weaknesses currently existing within most watermarking schemes. We also provide new insights into the reasons why adversaries will attack a watermarked model *despite accuracy loss* and discuss the reasons previous backdoor-removal techniques do not exploit the weaknesses in watermarks specifically.

2. Related work

2.1. Neural network backdoors

Backdoor attacks on neural networks have highlighted critical weaknesses of the black-box nature of neural networks throughout a variety of different tasks and model structures (Chen et al., 2017; Gu et al., 2017; Liu et al., 2018b). Backdoors essentially exploit the vulnerability of the overparameterization of deep neural networks to hide deliberately-designed backdoors in the model.

If the training of a network is outsourced to a third party that surreptitiously inserts specific and maliciously-labeled training images, the victim organization will receive a model that behaves correctly on the surface but contains a hidden backdoor. For example, the BadNets research (Gu et al., 2017) demonstrated that it was possible to force the misclassification of stop signs to more than 90% using only a yellow Post-it note sized square overlaid on the images.

2.2. Backdoor watermarking techniques

Some research has proposed utilizing this weakness to backdoor attacks as a method for embedding watermarks. Originally, Uchida et al. (2017) proposed embedding watermarks into the convolutional layer(s) of a deep neural network using a parameter regularizer. However, their procedure required white-box access to the neural network weights, which is not feasible or practical in many theft scenarios. Therefore, black-box watermarking models have been proposed as well, which we refer to as backdoor watermarks within this paper. One proposed model takes advantage of adversarial examples to detect watermarks (Merrer et al., 2017), but this method may be vulnerable to retraining techniques aimed to reduce the

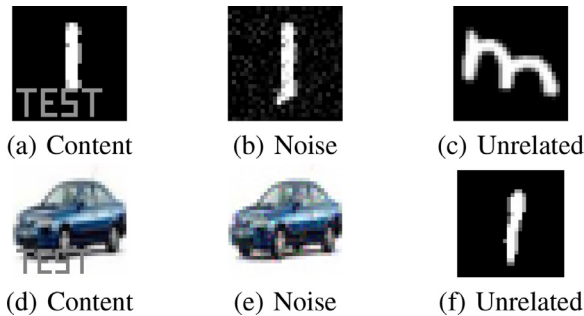


Fig. 1 – Examples of watermarked images used in the MNIST (top) and CIFAR-10 (bottom) datasets following the description in previous work (Zhang et al., 2018). MNIST examples are watermarked to be classified as “0”, and CIFAR-10 examples as “airplane”.

impact of adversarial examples, and is currently outside the scope of the paper.

More robust backdoor watermarking methods include Zhang et al. (2018)’s black-box technique that uses watermarked images as part of the training set of the network, consistently labeled as one class. These watermarked images include the following three types of images as shown in Fig. 1: (1) meaningful content (e.g., a word) placed over part of the image in some subset of training images (“content”), (2) pre-specified (Gaussian) noise over some subset of training images (“noise”), or (3) completely unrelated images (“unrelated”). For clarity, the “unrelated” watermarks, as well, are also all assigned to one class, either “0” or “airplane”, in MNIST and CIFAR-10, respectively.

Note that Fig. 1(f) is an image taken from MNIST but used as an “unrelated” watermark in CIFAR-10. In their experiments, they demonstrate minimal impact on the accuracy of the model, and their watermarks remain strong even after substantial pruning, tuning, and model inversion attacks (Fredrikson et al., 2015) against the watermarked model.

Adi et al. (2018) similarly use backdoors to prove non-trivial ownership. In their approach, the authors utilized 100 abstract images, each randomly assigned to a target class for their trigger set. Furthermore, their embedding procedure required the owner to sample k images from the trigger set during each training batch.

These approaches (Adi et al., 2018; Zhang et al., 2018) allow for the embedding and detection of watermarks that are human-readable (content-based watermarks) as well as those that are not human-readable (unrelated or noise-based watermarks) in black-box scenarios. Moreover, Zhang et al.’s model (Zhang et al., 2018) appears to be considered for deployment at large IT companies that deploy deep neural network services, such as IBM.¹ However, it is still questionable whether currently suggested state-of-the-art watermarking techniques are sufficiently robust against sophisticated and targeted manipulation of the structure of the neural network, as claimed in their proposals.

2.3. General backdoor removal techniques

Other research has tackled a similar problem of removing backdoors in neural networks, and one highly related research is the work by Liu et al. (2018a). This work proposed a two-step process to remove backdoors by first pruning the network followed by fine-tuning the pruned network. Their method shows success in removing backdoors from different deep neural network implementations, but in general, from a defenders point of view, fine-pruning alone impacts accuracy too much.

Similarly, recent research by Wang et al. (2019) has shown the ability to recover sufficiently similar backdoor triggers embedded in maliciously backdoored neural networks. Their research focuses on the ability of victims to detect and remove backdoors from their networks, where the victims have access to their full training set but would like to avoid intensive re-training on the model.

However, this approach does not consider the point of view of the attacker trying to steal a watermarked neural network, nor does it consider the “unrelated” type of watermark because they are less common as backdoors in general. For such offensive cases, an attack may have an extremely limited dataset because high-quality labeled data may be expensive or rare. Our approach considers this novel perspective and proposes ways to augment watermark removal when limited in data or facing “unrelated” watermarks.

2.4. Backdoor watermark removal techniques

Furthermore, two additional studies explore the ability to mitigate the robustness of watermarks that take advantage of the backdoors of neural networks. Hitaj and Mancini (2018) propose using an ensemble of networks to reduce the likelihood of a watermark being correctly classified as well as a detector network that attempts to return random classes if it believes a watermark is present in the image. Shafieinejad et al. (2019) also investigate content, noise, and abstract categories of watermarks, but instead of attempting to recover the watermark then remove it, the authors propose copying the functionality of the model directly through queries via a non-watermarked dataset.

Indeed, the attacks to copy the functionality of neural networks via prediction results of APIs and the defenses to such attacks are an open field of research; however, recent work by Atli et al. (2020) has shown that some model stealing approaches are currently very effective and nearly undetectable. As a result, we argue that a watermarking scheme must first be able to meet baseline requirements of the definition of an effective watermarking algorithm. Uchida et al. (2017) include requirements Robustness (that a model should be robust to modifications like fine-tuning) and Security (that the watermark cannot be discovered or modified). As of yet, the watermark approaches we evaluate within this paper do not seem to meet these standards that watermarking algorithms are required to meet, even without addressing the concerns of model extraction attacks.

Functionality extraction attacks can, moreover, be used in tandem with our approach, but our work focuses on the ability to remove the watermarks from the network parameters

¹ <https://www.ibm.com/blogs/research/2018/07/ai-watermarking/>

directly, demonstrating more fundamental watermarking algorithms are required.

3. Attack model

In this section, a generic attacker-defender scenario is described that would exemplify in which situations our proposed laundering algorithm would be used and by whom.

In our attack model, we define two parties: the true owner O of the neural network model m and the plagiarizer P who has managed to procure m . P may have acquired m through a variety of ways, not all of which may be malicious; however, the exact means by which P acquires m is outside the scope of this paper. The model m performs a particular task t , but is watermarked in such a way that certain, carefully-constructed examples X_w will give highly specific output at the task t . In our attack model, the goal of the plagiarizer P is to alter m in such a way that the examples X_w will no longer result in predictable outputs from m while minimally impacting t .

Importantly, our attack model places certain limitations on plagiarizer P . First, P has a substantially limited set of training data when compared to the creator O . Otherwise, P could trivially label a large set of non-watermarked training data using m to create a non-watermarked model by predictive model theft techniques (Tramèr et al., 2016), and this is not the goal of our algorithm. Second, P does not know if m has been watermarked but assumes it to be. Therefore, our proposed attack method should be able to overcome the robustness of the watermarked model m with limited training data such that m can adequately perform task t without reacting to watermarked examples.

As described in other work (Rouhani et al., 2019; Uchida et al., 2017; Zhang et al., 2018), watermarks in neural networks are designed to be robust to pruning, fine-tuning, and/or watermark overwriting as well as secure against the discovery of the presence of a watermark in the model. Although there exist watermarking techniques robust to both traditional pruning and retraining, we present a more sophisticated method that can significantly hinder and defeat the effectiveness of these black-box backdoor watermark embedding algorithms.

4. Proposed laundering technique

In this section, we describe our algorithms in more detail. Adversaries, which were described as the plagiarizer P in the previous Attack Model section, have access to the intermediate pre-trained layers of the watermarked neural network (L_j), which, in this case, have been watermarked during the original training process. Additionally, the adversaries have their own training dataset that has been labeled correctly (X_i). Adversaries have procured this correctly-labeled dataset either manually or perhaps by using the watermarked network's outputs to label the examples automatically. It is important to note that the watermarked neural network's output would not intentionally misclassify any of these images during the creation of the correctly-labeled dataset if the adversary chooses that method because none of these images would contain such a specific watermark by pure chance.

The novel contributions of our complete algorithm, specifically designed to help function in black-box, backdoor watermark-removal scenarios, include:

- Combining the “pruning” and “unlearning” steps proposed in Wang et al. (2019) as a two-step approach to removing watermarks even with very limited retraining data.
- Implementing a statistical approach used to decide if neurons should be reset within a layer based on the relative average activation of the entire dense or convolutional layer.
- Extensively investigating “unrelated” and “noise” styles of watermarks, which tend to avoid detection and removal in previous backdoor removal schemes (Liu et al., 2018a; Wang et al., 2019).
- Offering an additional application of the reverse-engineered masks generated during the watermark reverse-engineering process to aid in the removal of the unrelated style of watermark.

In order to perform an attack on neural network backdoor watermarks and achieve the above listed contributions, we propose the following 3-step process: (1) watermark recovery, (2) black-box attack laundering via the algorithm proposed in Algorithm 1, and (3) black-box adversary retraining. Step 2 is

Algorithm 1 Watermark laundering algorithm (Step 2).

```

1: Given:
   Trained intermediate layers ( $L_j$ ) for  $j = 0, 1, \dots, J$ 
   Normally labeled data ( $X_i$ ) for  $i = 0, 1, \dots, I$ 
   (Recovered) Watermarked data ( $X_k$ ) for  $k = 0, 1, \dots, K$ 
2: Predefined:
   Dense layer activation threshold (DT)
   Convolutional layer activation threshold (CT)
3: for  $j = 0, 1, \dots, J$  do
4:   for  $i = 0, 1, \dots, I$  do
5:      $AN_j^{total} += L_j(X_i)$ 
6:   for  $k = 0, 1, \dots, K$  do
7:      $AW_j^{total} += L_j(W_k)$ 
8: Avg layer watermark activation  $AW_j^{avg} = AW_j^{total} / K$ 
9: Avg layer normal activation  $AN_j^{avg} = AN_j^{total} / I$ 
10: Activation difference  $A_j^{diff} = AW_j^{avg} - AN_j^{avg}$ 
11: for  $j = 0, 1, \dots, J$  do
12:   if  $L_j$  TYPE is DENSE then
13:     for  $v = 0, 1, \dots, V$  do
14:       if  $AN_{jv}^{diff} > DT$  then
15:         Reset intermediate layer neuron  $L_{jv}$ 
16:   else if  $L_j$  TYPE is CONV then
17:     for  $v = 0, 1, \dots, V$  do
18:       if  $AN_{jv}^{diff} > CT$  then
19:         Reset intermediate layer channel  $AN_{jv}$ 

```

required to reset potentially watermarked neurons, and Step 3 restores the performance of the fully laundered model back to acceptable levels in instances where Step 2 has reset a large number of neurons that also serve an important role in final non-watermarked classification. We present the end-to-end black-box laundering procedure in Fig. 2. We will now go more in-depth on each of these steps.

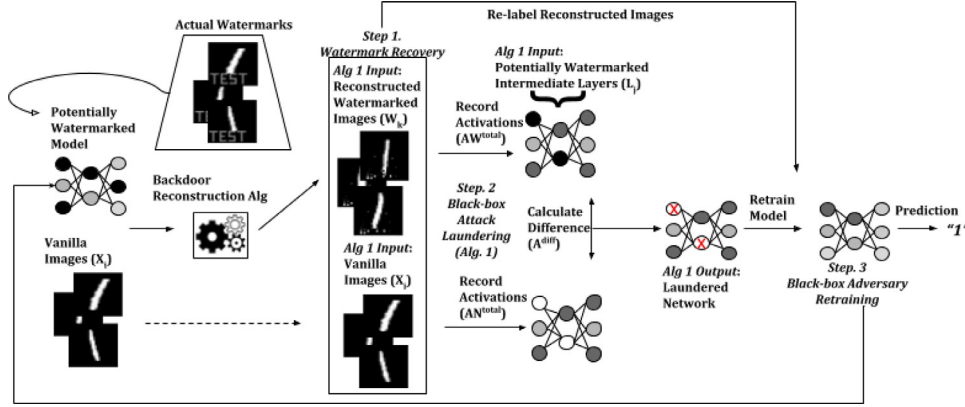


Fig. 2 – Overview of the black-box laundering procedure, where the vanilla images are normally labeled data and red crosses are the reset neurons. This shows the process of recovering potential watermarks, laundering the network via our algorithm multiple times, and finally retraining the network based on re-labeled reconstructed images. The process is repeated over multiple iterations where the retrained model is fed back into the reconstruction algorithm.

Step 1. Watermark recovery. As discussed in Section 2, there exist methods to discover potential backdoors within neural networks (Wang et al., 2019). We use such a method to reconstruct outlier perturbations required to push one class to another. Specifically, their method includes the following formulation:

$$A(x, m, \Delta) = x' \\ x'_{i,j,c} = (1 - m_{i,j}) \cdot x_{i,j,c} + m_{i,j} \cdot \Delta_{i,j,c}, \quad (1)$$

where $A(\cdot)$ is the trigger application function, x is the original image, Δ is the trigger image, and m is the mask for the trigger. The authors also factor in the magnitude of the trigger by the L1 norm of m to result in the final formulation of:

$$\min_{m, \Delta} \ell(y_t, f(A(x, m, \Delta))) + \lambda \cdot |m| \\ \text{for } x \in \mathbf{X}, \quad (2)$$

$f(\cdot)$ is the networks output prediction function, $\ell(\cdot)$ is the loss function, λ controls the magnitude for controlling the size of the reversed trigger, and $\mathbf{X}$ represents the available non-watermarked images. For more detailed information regarding

the construction scheme, we direct the reader to the original paper (Wang et al., 2019) or to their open-source implementation²

However, this step alone is insufficient in the watermark removal domain, particularly due to the existence of “unrelated” and “noise” types of watermarks.

Step 2. Black-box attack laundering. While adversaries to backdoor watermarks have no access to known watermark images, they assume the reconstructed watermarks from Step 1 to be accurate representations of the actual watermarked images. As such, the reconstructed watermarks are overlaid on the adversary’s available training data, and these sets are sent through the laundering algorithm.

Using these manually-constructed watermarks, the adversary can observe and record the activations of each layer L_j for

each watermarked image in W_k of the neural network to calculate the total watermarked activation for each neuron AW_j^{total} .

Then, the adversary simply calculates the average activation of each neuron individually AW_j^{avg} by dividing by the number of training examples, as shown in line 8 of Algorithm 1.

Following this, the adversary may perform a similar calculation across all non-watermarked images X_i through all the neural network layers L_j and observe and record these observations for the total non-watermarked activation for each neuron AN_j^{total} . The adversary then calculates the average of each neuron to calculate AN_j^{avg} . Consequently, the adversary now has the ability to subtract the average normal image activation AN_j^{avg} from the average watermarked activation AW_j^{avg} . This results in the average difference in activation of that layer A_j^{diff} demonstrated in line 10 of Algorithm 1.

From here, the adversary simply resets any neuron that activated strongly when in the presence of watermarked images but did not activate strongly for non-watermarked images. The adversary does this by stepping through each neuron v in L_j and removing it if the difference in the activation between watermarked and non-watermarked images (A_{jv}^{diff}) falls above some threshold value DT as in line 14 of Algorithm 1. If the layer is convolutional (and the activation difference falls above CT), then the adversary would instead reset the entire intermediate channel AN_{jv} .

“Resetting” a neuron or channel may take many forms. In our case, we immediately reset the weights of the input into the layer to zero during the algorithm, maintaining those reset weights on each successive pass through all layers L_j in order to retrain the network, while preventing the watermarked neurons from reappearing. For most activation functions, setting the weights to zero would achieve the desired effect; however, this may not always be the case. Neurons that leverage activation functions such as softplus (Dugas et al., 2001) that produce non-zero output at point 0 may still be highly activated in the presence of zero-weighted inputs. In such a case, it would be up to the adversary to choose a more appropriate resetting procedure. Moreover, Liu et al.’s Fine-

² <https://github.com/bolunwang/backdoor>.

Pruning (Liu et al., 2018a) proposes that special training can reduce the impact of neutralizing a neuron such as we propose within this step, and so we must acknowledge that watermarking schemes that do implement those procedures will be able to mitigate the impact of this step specifically. For that reason, we propose the final deliberate unlearning step below.

Step 3. Black-box adversary retraining. Finally, the model is retrained on all available examples, including those collected during reconstruction. Note that these non-watermarked examples were the same examples passed through the neural network in the laundering step, except in this step, watermarked examples are labeled as the correct class. This is similar to retraining steps in backdoor-removal methods (Liu et al., 2018a; Wang et al., 2019); however, unlike backdoor scenarios, we also face the “unrelated” watermark which has no “correct” label. Also, unlike Neural Cleanse (Wang et al., 2019), we use the Median Absolute Deviation technique to identify the class least likely to be watermarked and label our reconstructed unrelated images to that class during retraining. We can calculate the least likely class within Wang et al.’s method (Wang et al., 2019) because we can find which class has the least likely chance of being an outlier when dividing the absolute deviation of all data points by the median absolute deviation, where the median absolute deviation is simply calculated by $\text{median}(|X_i - X_{\text{avg}}|)$. This is another important novel contribution of our work and distinguishes us from other work with similar goals.

For this reason, we stop laundering early if the original most-likely-infected class is considered to be the least likely to be infected. Otherwise, retraining could be strengthening the original watermark. Additionally, we also feed the reconstructed masks themselves generated during the reverse-engineering step into the retraining dataset. These are also labeled as the least likely class, which acts as a secondary approach to removing neurons that are potentially watermarked with the unrelated style of image. The retrained model is then fed back into the backdoor reconstruction algorithm.

5. Experiments

In order to demonstrate the feasibility of our approach, we recreated the results of Zhang et al.’s work (Zhang et al., 2018) for both MNIST and CIFAR-10. In general, the architectures of the deep neural networks followed the procedure according to the original papers (Adi et al., 2018; Zhang et al., 2018). We followed Zhang et al.’s watermarking implementation as described in their paper (Zhang et al., 2018) for MNIST and CIFAR-10. However, for CIFAR-10, the original network architecture used in Zhang et al.’s work (Zhang et al., 2018) was insufficient to reach approximately 90% accuracy at the classification task. As a result, we used a modern architecture that more appropriately reflects the expected performance on the CIFAR-10 task, which explains the larger discrepancies within Zhang et al. on CIFAR-10. Moreover, because we trained the models ourselves, the un-laundered test accuracies and un-laundered watermark accuracies are not exactly identical to the original work. Similarly, for re-implementing Adi et al.’s watermarking scheme, we followed both the pre-trained (PT) procedure where watermarks are embedded into the network fol-

lowing non-watermarked training as well as the from-scratch (FS) procedure where watermarks are embedded during training (Adi et al., 2018).

The results in this section correspond to **one round of laundering**. We reset all layers in all DNNs except for those models using ResNet18+ (which is used in Adi et al.’s scheme Adi et al., 2018). Due to the depth of that model, we reset the second half of the weights only; the weights of the first half appear to be learning very high-level features that do not correspond directly with the watermarks, as would be expected in such models that frequently used in transfer-learning scenarios.

We implemented our neural laundering prototype in Python 3.6 with Keras 2.1.6 and Tensorflow 1.7.0. The experiments were conducted on a machine with an Intel i5 CPU, 16 GB RAM, and 3 Nvidia 1080 Ti GPUs with 11 GB GDDR5X.

To evaluate our laundering technique in a realistic setting, we limited the adversary’s retraining dataset size. Especially for MNIST, using even half of the test set size for training or retraining as performed in Zhang et al.’s original work (Zhang et al., 2018) is sufficient to train a model to above 90%, with or without the watermarked model. As a result, if we do not limit the training size, adversaries could simply train their own neural network from the output of the watermarked model using prediction algorithms found in Tramèr et al.’s work (Tramèr et al., 2016). This situation inherently implies there would be no need for laundering a watermarked network at all, given a large retraining set size, and this issue is frequently overlooked in related work.

As a result, for the results against Zhang et al.’s watermarking scheme (Zhang et al., 2018), we purposely limited the adversary’s MNIST retraining dataset to be 0.6% of the original training set size of 60,000 handwritten digits, which results in approximately 42 images per category. Likewise, we also limited the CIFAR-10 dataset to 0.6%, which results in approximately 36 images per category. In reality, adversaries would likely have more data available and achieve better results than the conservative results we report.

For the following evaluation sections, we will repeatedly refer to Table 1, wherein we list our results - in rows following “Ours” in column 1 (“Method”) - directly below the proposed watermarking approach and other authors’ original watermark removal attempts for comparison. Column 4 (“Un-laundered Test Accuracy”) and column 5 (“Laundered Test Accuracy”) record the accuracy on the never-before-seen test set by the original watermarked network before laundering and then subsequently on the laundered watermarked network, respectively. Similarly, column 6 (“Un-laundered Watermark Accuracy”) and column 7 (“Laundered Watermark Accuracy”) record the accuracy of the detected watermarks by the original watermarked network before laundering and then subsequently on the laundered watermarked network, respectively. The thresholds at which the watermarks are unusable for ownership demonstration purposes are listed in column 8 (“Vanilla Model Watermark Accuracy”), and we discuss how we derive these values in Section 7.1. Finally, we provide a comparison of the size of the dataset used in the watermark removal process in column 9 (“Limited Retraining Size”), which corresponds to the “normally labeled data” referenced in Algorithm 1. We also include a “Vanilla Model Test Accuracy” column to demonstrate that we follow the neu-

Table 1 – Results of the black-box adversary algorithm for various watermarks and backdoors.

Method	Dataset	Watermark type	Vanilla model test accuracy	Un-laudered test accuracy	Laudered test accuracy	Un-laudered watermark accuracy	Laudered watermark accuracy	Vanilla model watermark accuracy	Limited retraining size
Zhang et al. (2018)	MNIST	Content	99.28%	99.46%	97.03%	100%	99.95%	1.5%	16.6%
(90% pruning)		Noise	99.28%	99.41%	95.19%	100%	99.55%	6.0%	16.6%
		Unrelated	99.28%	99.43%	93.55%	100%	99.9%	20%	16.6%
Ours		Content	99.31%	99.90%	98.34%	100%	0.01%	1.5%	0.6%
		Noise	99.31%	99.86%	97.45%	99.99%	0.07%	6.0%	0.6%
		Unrelated	99.31%	99.92%	98.33%	99.71%	15%	20%	0.6%
Zhang et al. (2018)	CIFAR-10	Content	78.6%	78.41%	64.9%	99.93%	99.47%	5.0%	20.0%
(90% pruning)		Noise	78.6%	78.49%	59.29%	100%	65.13%	4.0%	20.0%
		Unrelated	78.6%	78.12%	62.15%	99.86%	10.93%	52.0%	20.0%
Ours		Content	77.8%	90.24%	87.65%	100%	1.4%	5.0%	0.6%
		Noise	77.8%	89.01%	84.14%	100%	0.50%	4.0%	0.6%
		Unrelated	77.8%	89.77%	85.34%	99.94%	16%	52.0%	0.6%
Adi et al. (2018) PT	CIFAR-10	Unrelated	93.42%	93.65%	~90%	100%	100%	7%	~
Ours (PT)			92.15%	91.55%	88.25%	100%	7.0%	12%	10.8%
Adi et al. (2018) FS			93.42%	93.81%	~90%	100%	80%	7%	~
Ours (FS)			92.15%	91.85%	84.73%	100%	7.0%	12%	10.8%

ral network architecture and training procedures truthfully. Note that for Zhang et al. (2018)'s and Adi et al. (2018)'s CIFAR-10 models, our vanilla architectures routinely converged slightly below their reported test accuracy (0.8% and 1.3%, respectively) despite repeating the training procedure multiple times.

5.1. MNIST and CIFAR-10 results via Zhang et al.'s method

For the MNIST model across all watermarking types, our method was able to remove detected watermarks below unusable levels. The content and noise watermarks are significantly removed. They fall significantly below even random classification, and although unrelated watermarks are much higher (e.g., 15% for the MNIST unrelated watermark), they fall below the vanilla model watermark accuracy (e.g., 20% again for the MNIST unrelated watermark).

Again, for CIFAR-10 as well, our method was able to remove detected watermarks below usable levels for all watermark types, while minimizing the drop in test classification to about 5% overall. One point of importance is that for CIFAR-10 unrelated watermarks; our model does not reduce watermark accuracy as much as the reported pruning techniques used in Zhang et al. (2018). Nevertheless, our method does maintain a higher percentage of the final test accuracy. The original paper recorded a drop from 78.12% accuracy to 62.15% (see Table 1) while our method results in a drop from 89.77% accuracy to 85.34%. Additionally, our method demonstrates its effectiveness even when using a smaller dataset size.

5.2. CIFAR-10 results via Adi et al.'s method

In Adi et al.'s proposed watermarking scheme (Adi et al., 2018), they included 100 unrelated (abstract) images, labeled each

image with a random class, and trained their model on these images as well. However, their method proposed two ways to embed the watermark, either into the model from scratch (FS) or by fine-tuning a pre-trained (PT) model. We include evaluations of both approaches in Table 1.

While there are perhaps more efficient ways to attack the Adi et al. (2018) watermarking scheme (e.g. an out-of-distribution detector), we do not adapt our method to be specifically tailored to the Adi scheme. We aimed to test our watermark removal strategy as an agnostic method with only one minor modification. Empirically, for very deep networks such as ResNet18+ (He et al., 2016), resetting many shallow layers significantly impacted the overall final performance of the model. While one layer is not enough, resetting all shallow layers results in deep layers receiving activations that they were not originally trained to receive. As a result, for the ResNet model, we chose to reset only the second half of the weights. Additionally, the results in Table 1 include a larger subset of available training data than used against Zhang et al. (2018). A further discussion of this is explained in Section 6.

We find that in both cases (during training and post-training), the watermarks are significantly less robust than the original claims by Adi et al. (2018) using our approach. We compared our results to the original watermark removal technique referred to as "Re-train All Layers (RTAL)". Even in the original research, the RTAL watermark removal technique had significantly reduced the accuracy of the pre-trained watermark set. However, we also find that the FS watermarks face similar problems, perhaps due to the reliance on batch normalization (Ioffe and Szegedy, 2015). Before retraining, we also require the batch normalization layers to relearn their gamma and beta parameters. Additionally, the authors do not use a limited dataset to evaluate their watermark removal methods.

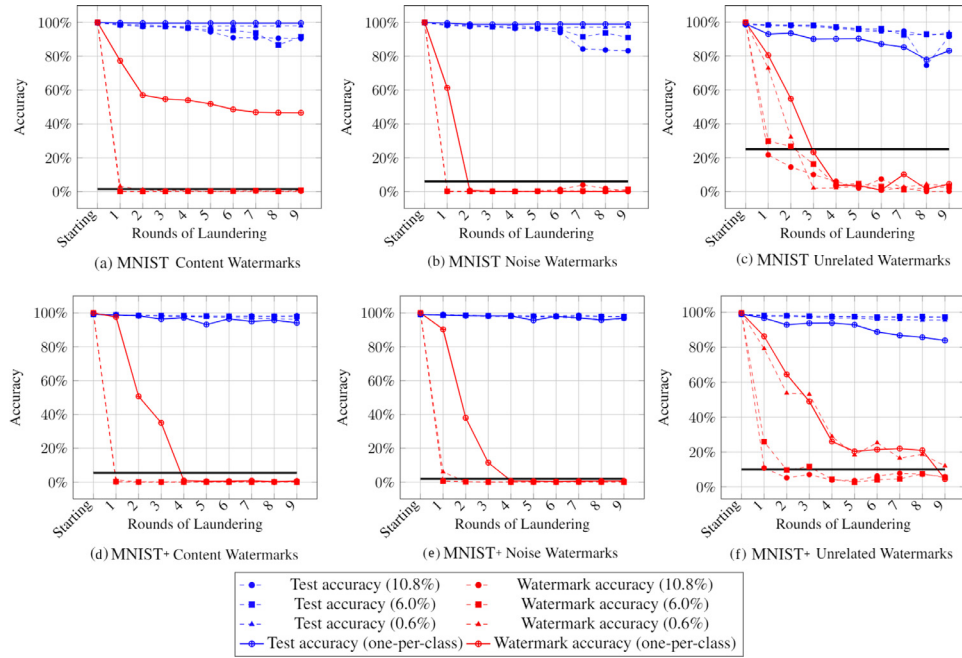


Fig. 3 – Results of laundering the MNIST and MNIST+ models to remove Zhang et al.'s proposed watermarks (Zhang et al., 2018).

6. Evaluation of required data for watermark removal

In other backdoor or watermark research (Liu et al., 2018a; Zhang et al., 2018), researchers have typically given the remover a train-test split worth of data. For example, in the original Zhang et al. (2018) watermark proposal, the adversary was given the entire test set of the default train-test split from the MNIST dataset to attempt to remove the watermark via pruning and retraining. Even though the removal techniques discussed in Zhang et al. (2018) are unable to remove watermarks, an adversary would not need to remove them at all given that size of a dataset. Using the neural network structure and dataset split utilized in Zhang et al.'s paper (Zhang et al., 2018), adversaries could train their own MNIST models using the test set only and still easily reach 90% or above accuracies, completely removing the need to launder the watermarked model. However, because our approaches can directly target weaknesses of watermarking techniques and because we approach this issue from the point of an attacker, we may consider the adversary to be even more limited than the other watermark or backdoor removal research has considered.

Therefore, in the above Table 1 in Section 5, we included the results where an adversary was limited to 0.6% (except in the case of Adi et al., 2018, shown in the final section of Table 1) of the total training data as well as the results from only one round of laundering. However, we also investigated the effectiveness of our approach under a variety of other splits. These include 10.8%, 6%, and 0.6%, and finally only one example per class, as depicted in the various lines in Figs. 3 and 4. Additionally, because a black-box adversary has no knowledge of

the watermark detection accuracy, we also evaluated scenarios where an adversary performed multiple rounds of laundering (up to 10).

In Figs. 3 and 4, the blue lines represent the final test accuracy on unseen data, and the red lines represent the accuracy on watermarked images. Each line style is representative of available dataset size, and the solid black line represents what we propose to be the minimum watermark accuracy required to claim ownership of a model. We discuss the minimum watermark accuracy value more in Section 7.1.

6.1. MNIST results via Zhang et al.'s method

Due to the simplicity of the model, even a small number of laundering examples - down to even one image per class in some cases - is enough to make a significant impact on the watermark accuracy (while maintaining test accuracy), especially with a small number of iterations. We show that most combinations perform similarly in Fig. 3. For example, Fig. 3(a)–(c) all show a decline in watermark accuracy (via red lines) after a few rounds. Additionally, the final testing accuracy (via blue lines) shows significant resilience even after multiple rounds of laundering, gradually declining over time. However, perhaps due to the complexity of the watermark in the content-based scenario, as shown in Fig. 3(a), one example per class (solid blue and red lines) is not enough to remove the watermark for plain MNIST.

Moreover, we also trained a more complex version of MNIST with six additional classes taken from the EMNIST dataset (Cohen et al., 2017) (specifically letters 't', 'u', 'w', 'x', 'y', 'z'). Note that while these images are from the EMNIST dataset, we refer to this model as the MNIST+ model as it does not contain all EMNIST classes. As shown in Fig. 3(d)–(f), the

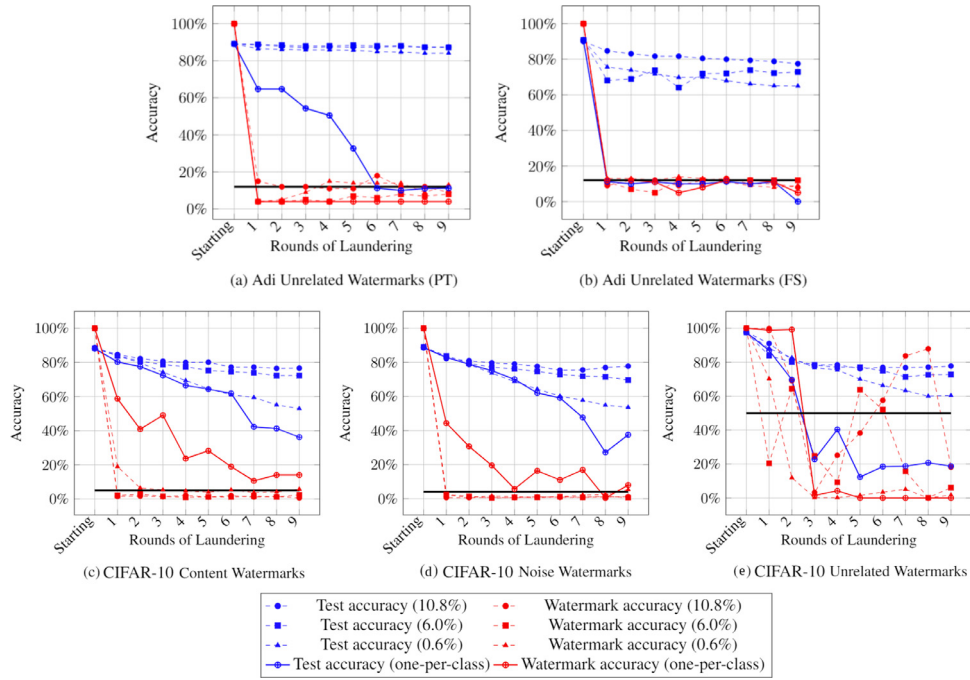


Fig. 4 – Results of laundering the CIFAR-10 model to remove Adi [Adi et al. \(2018\)](#)’s and [Zhang et al. \(2018\)](#)’s proposed watermarks.

inclusion of additional classes made two notable differences. First, in this model, our algorithm was able to remove the content-based watermarks completely. This strongly suggests that having a few more training examples is enough to reconstruct content-based watermarks more effectively, although future work will be required to understand this phenomenon in depth. Second, because of the larger class size, the unrelated watermarks were classified as the watermarked class less often in the clean network, pushing the target black line lower.

As a result, the smaller dataset sizes (0.6% and one-per-class – dashed triangles and solid lines, respectively) struggled to push the watermark below that threshold. Our MNIST+ dataset captures more generalized results than plain MNIST in the testing accuracy as well. As expected, one-per-class impacted testing accuracy the most. Due to the simplicity of plain MNIST, this phenomenon was not captured in those experiments.

6.2. CIFAR-10 results via Adi et al.’s method

Although the CIFAR-10 task is significantly more complex than MNIST, the 10.8%, 6.0%, and 0.6% retraining sets in [Fig. 4\(a\)](#) and (b) were able to remove the watermarks, while maintaining a high test accuracy. However, as expected, the accuracy dropped over multiple rounds of laundering, particularly as the size of the laundered set decreased. We speculate that this is owed more to overfitting than to our laundering method. In this case, however, the one-per-class laundering set is entirely inadequate, especially over time, in both cases. Regardless of the combination, the [Adi et al. \(2018\)](#)’s from-scratch (FS) technique outperforms the pre-trained (PT) technique. Due to the complexity of the task and the resilience of the watermarking scheme, our scheme does incur a loss of

test accuracy on small laundering set sizes. We discuss the implications of this drop in test accuracy in [Section 7.3](#).

6.3. CIFAR-10 results via Zhang et al.’s method

Again due to the complexity of the task required to classify CIFAR-10 images, we expected a large impact on the test accuracy over time, and the results do show this phenomenon in [Fig. 4\(c\)–\(e\)](#). As in the Adi et al. model, the one-per-class laundering set is again inadequate. However, both the content and noise watermarks were removed via our method in the 10.8%, 6.0%, and 0.6% cases, although the cost to final test accuracy worsened across multiple iterations. The unrelated watermarks were much more difficult to consistently remove, with seemingly random high fluctuations. However, the unrelated watermarks in CIFAR-10 are also classified randomly even in vanilla models, and we argue that relying on such a high degree of uncertainty of a watermark is quite risky, and we go into more detail about these risks in [Section 7](#).

6.4. General laundering observations

To conclude this rather extensive evaluation, we make general observations regarding the rounds and percentages used for laundering:

1. The larger the laundering dataset size, the more rounds of laundering are possible without a subsequent decrease in accuracy.
2. The more complex the model, the more likely subsequent rounds of laundering will significantly harm the final test accuracy. This is the price to pay for trying

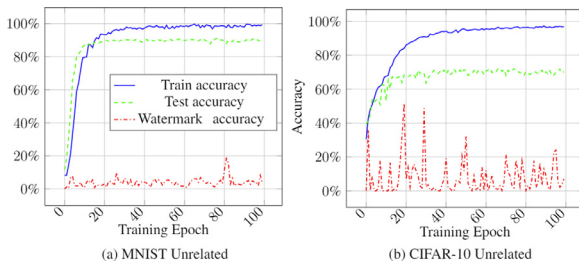


Fig. 5 – Demonstrations of how legitimately trained models can fluctuate on classifying a watermarked image as the backdoored class during training.

to remove a watermark entirely from a neural network more confidently.

3. In most cases, once the watermark has been removed, it is unlikely to return unless it was embedded with a high randomly-occurring threshold (as in CIFAR-10 unrelated watermarks).

As a result, we make a general recommendation rule that laundering more than three rounds will result in diminishing returns for adversaries where they will be losing final test accuracy without removing additional watermark accuracy.

7. Discussion

In this section, we discuss several key issues related to choosing watermark detection thresholds, the integrity and reliability of recovered watermarks, and other important considerations of our approach.

7.1. On choosing the minimum watermark detection threshold

Fig. 5 (a) represents our MNIST model training on clean data and attempting to classify unrelated watermarks. As shown in Fig. 5, there is no true final value at which a converged, clean model will classify “unrelated” watermark images. It is entirely possible that a legitimate network could stop being trained during the epoch where the watermark classification accuracy is quite high, purely by coincidence. For the unrelated watermarks and especially for the CIFAR-10 model in Fig. 5(b), this effect is even greater, with fluctuations reaching above 50% after certain epochs.

To expand further, inputting random images beyond the scope of a neural network’s training may likely result in unpredictable output. For example, a model trained on CIFAR-10 data should not have an expected output when inferencing an image from the MNIST dataset. In our experiments, some (but not all) completely vanilla CIFAR-10 models would misclassify the “1” image from MNIST as “airplane” with high confidence. As a result, even if a watermarked network has been deliberately trained to perform in this way, because a non-watermarked model could potentially return similar outputs, we believe that this does not demonstrate ownership of the model. Any laundered network that falls below the high-

est naturally occurring watermark classification rate is unable to provide any provable ownership for the true owner of the network. Furthermore, even laundered models that fall closely above this threshold may not provide any meaningful ownership for the true owner of the network because a watermarked model relies strictly on the accuracy, not loss, and a (perhaps random) fluctuation of a couple of percentages may not be enough to demonstrate ownership of a model. Note that in Fig. 5, the CIFAR-10 model does not reach 90% because we did not perform data augmentation (rotating, scaling, etc.) during the training of this example model.

7.2. On the reliability and integrity of DNN watermarks

In the DeepSigns (Rouhani et al., 2019) research, the authors further expand upon the requirements of effective watermarks proposed in Uchida et al.’s original design (Uchida et al., 2017). In addition to including generalizability (that the model should work in both white-box and black-box settings), the authors argue that neural network watermarks should have both 1) **reliability** to yield minimal false negatives and 2) **integrity** to yield minimal false positives. However, using our method on watermarked neural networks significantly hinders and defeats the ability of current watermark embedding processes to claim reliability and integrity. Adversaries can significantly increase false negatives and false positives in watermarked networks without access to the original training watermarks or the original training set.

For example, the reliability and integrity of the watermarks are in question if the detection accuracy is not significantly consistent. We argue this is especially true in unrelated watermarks because unrelated watermarks have no “correct” answer in these networks. In the MNIST set, the network was trained to classify the letter “m” as a “0”. After applying adversary laundering, the unrelated images were still classified as “0” 16% of the time, but were classified as “8” 45% of the time and “9” 25%. In the case of CIFAR-10, the unrelated images of an MNIST “1” digit were embedded as the watermarked class of “airplane” and were detected as such 23% of the time, but were also classified as “ship” 52%, and as “truck” 21%. Non-watermarked networks could come to similar classifications and may not demonstrate ownership in the current methods.

7.3. On the drop in test accuracy of laundered models

In proposed watermarking strategies (Zhang et al., 2018), the proponents could argue that the drop in test accuracy from watermark removal techniques is sufficient to prevent attackers from stealing or using stolen models. In our examples, our method generally performs well regarding the ability to maintain test accuracy, with the largest drop occurring in CIFAR-10 against the Zhang et al. (2018) “unrelated” watermark type of approximately 9%. However, we argue that such a reduction (or larger) in accuracy does **not** imply that a laundered model is not useful. Indeed, while it is fair to point out that laundered accuracies on MNIST and CIFAR-10 do not reach either the original model accuracies nor the state-of-the-art, in many of our scenarios, an attacker would not be able to approach such accuracies with their limited datasets. We emphasize that a laundered model is useless only if the laundered

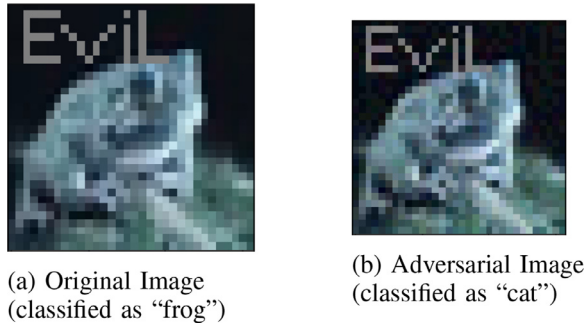


Fig. 6 – Adversarial example constructed against the stolen model that fools both the original model and the laundered model into classifying the image as “cat”.

model performs worse than a model trained from scratch on the same (limited) dataset.

In situations where adversaries have very limited datasets (e.g., 0.6% of the original training size), adversaries can use a stolen laundered model to improve their classification accuracy. In such cases, where training data may be challenging to obtain or is very expensive (such as medical data, high-quality clean data, etc.), a stolen model, even if watermarked, remains enticing to an adversary not only to steal for private use but also perhaps to launder and deploy.

Additionally, we also demonstrate that the reported results of black-box backdoor watermark attacks are overestimating the model's ability to retain those watermarks. Related work does not adequately explore the potential for attackers to detect and remove neurons and weights that are contributing to the detection of watermarks.

7.4. On the issue of adversarial watermarks

Another point to consider is that a stolen model is susceptible to a wide range of adversarial attacks. Notwithstanding other security threats posed by adversarial examples in such a scenario, given white-box access to a model, an adversary could easily manufacture images that they claim to be watermarked but are adversarially-crafted against the original network. In this case, it will be additionally difficult to prove ownership of the model.

For example, Fig. 6 contains an adversarial image constructed against a stolen Zhang et al. (2018) CIFAR-10 content watermarked network. The original model classifies the adversarial image of “frog” as “cat”. Due to the similarity between the two networks and the ability for adversarial examples to transfer between models (Tramèr et al., 2017), it is also misclassified as “cat” in the post-laundering model as well without targeting that model specifically. This particular laundered model performs with 85% accuracy on the original task, but with 1.5% accuracy on the original watermarks. However, when constructing adversarial images, the adversary can choose any percentage of final watermark detection accuracy by also purposefully crafting adversarial watermarks that fail to be detected.

Moreover, even without embedding their own “evil” watermark into this model, an adversary can simply create an

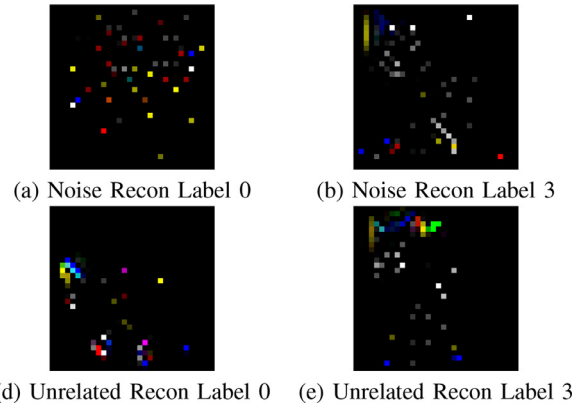


Fig. 7 – CIFAR-10 reconstructed watermarks using the Neural Cleanse (Wang et al., 2019) algorithm. Outlier detection struggles to detect the watermarked class; actual watermarked class is Label 0.

adversarial example that looks sufficiently similar to a watermark of which the adversary claims ownership. One may argue that adversarial examples of this sort may be easy to detect because the text (“evil”, in this case) contains some human-noticeable noise. However, an optimization algorithm that forbids noise in those pixel locations would likely find a suitable minimum without significantly more work. We leave such an implementation for future work.

7.5. Limitations of prior watermark removal techniques

Considering the similarities between watermarks and backdoors, it is natural to question why the original Neural Cleanse approach (Wang et al., 2019) is not sufficient for some backdoor removal attempts. We argue that this arises due to two differences in our approaches. First, our method does not inherently rely on the outlier detection mechanism to be accurate; an adversary assumes there to be a watermark regardless of its outcome. This is especially necessary for watermark removal due to 1) the limitation of the datasets available, and 2) the types of augmentations that are used as watermarks. Indeed, in the content-based watermark reconstruction, we do find that the results are as expected from the original approach; however, for the noise and unrelated reconstructions shown in Fig. 7, an outlier detection scheme may struggle to decide which label is watermarked.

Second, as described earlier, the unrelated watermark has no ground-truth class, and the original Neural Cleanse approach (Wang et al., 2019) and even Fine-Pruning (Liu et al., 2018a) do not consider this situation. Both of these two methods leave the unrelated watermark largely untouched. On the other hand, we retrain on all images applied with the reconstructed mask labeled as the correct class as well as on all masks themselves labeled as the least-likely class.

7.6. Application on backdoor attacks specifically

Throughout this work, the focus has been primarily on watermarks embedded into neural networks. However, most watermarking techniques can be considered a subset of backdoor-

Table 2 – Results of the black-box adversary algorithm against YT-Faces backdoor.

Method	Dataset	Watermark type	Un-laundered test accuracy	Laundered test accuracy	Un-laundered watermark accuracy	Laundered watermark accuracy	Vanilla model watermark accuracy	Limited retraining size
Fine-Pruning	YT-Faces	Content	97.4%	97.7%	99.8%	0.00%	0.00%	10%
Ours			99.04%	97.00%	100%	0.00%	0.00%	0.6%

ing attacks. As a result, we will focus on some of the implications of our proposed techniques specifically on backdoor attacks throughout the following subsection.

Neural network backdooring background Inserting backdoors into neural networks is a common threat against deep learning systems, and it typically occurs before initial training by data set poisoning. Sometimes referred to as a “trojaned” model (Liu et al., 2018b), deploying a network embedded with a backdoor provides an attacker with a means to easily and predictably control the output of a neural network. As long as the backdoor does not significantly affect the performance on the validation set, detecting the presence of these attacks can be very challenging due to the inherent black-box nature of neural networks. Backdoors have been demonstrated across a wide variety of tasks and network architectures (Chen et al., 2017; Gu et al., 2017; Liu et al., 2018b).

One such approach to removing backdoors in deep neural networks is the work by Liu et al. (2018a). The authors propose a two-step process to remove backdoors by first pruning the network followed by fine-tuning the pruned network. Their method shows success in removing backdoors from different deep neural network implementations. One such example is face identification using DeepID by Sun et al. (2014) backdoored by Chen et al.’s sunglasses content overlay (Chen et al., 2017). For these two networks, their results revealed that their method reduces the attack accuracy to 0% in the face identification task.

Because Liu et al.’s fine-pruning (Liu et al., 2018a) does not target watermarks specifically, in order to demonstrate the transferability of removing backdoors to removing watermarks in general, we also implement these network architectures and backdoor them. For evaluation, we consider basic backdoor poisoning to be a watermark embedded into the network, and we apply our watermark-removal techniques on them (using a limited amount of retraining data as done against watermarks above). Examples of the backdoors are shown in Fig. 8. Image (a) is a clean example of William Macy from the YouTube Faces (Wolf et al., 2011) dataset, and (b) represents a content-based backdoor for a facial recognition test, where all sunglasses images are to be labeled incorrectly as A. J. Cook.

YT-Faces results via Chen et al.’s backdoor method While the technique from Chen et al.’s method (Chen et al., 2017) is proposed as a backdooring technique, we investigate the effectiveness of our approach as a way to remove the backdoor and compare our results to the results of fine-pruning (Liu et al., 2018a). We implement the same backdoor trigger and neural network structure as utilized in the “Fine-Pruning” pa-

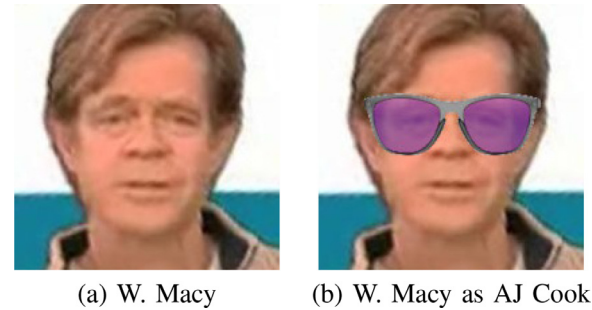


Fig. 8 – Additional examples of backdoors adopted in fine-tuning (Liu et al., 2018a) that we attempt to remove as well using laundering.

per (Liu et al., 2018a), and we are successfully able to match their findings on the YT-Faces dataset, and as with the other examples, we use a significantly smaller dataset in our model. A summary of these results is presented in Table 2.

8. Conclusion and future work

We proposed a novel “laundering” algorithm that focuses on low-level manipulation of a neural network based on the relative activation of neurons to remove the detection of black-box backdoor watermarks. We demonstrated significant weaknesses in current watermarking techniques, especially when compared to the results reported in the original research (Adi et al., 2018; Zhang et al., 2018). Specifically, we were able to remove watermarks below ownership-proving thresholds, while achieving test accuracies above 97% and 80% for both MNIST and CIFAR-10, respectively, for all evaluated model and watermark combinations using a highly restricted dataset (0.6% of the original training size in some cases).

In addition, we provided new insight into marrying the fields of watermark embedding and removal with backdoor embedding and removal. As we have shown in this research, these two will have opposite repercussions: increasing defense against backdoors reduces the effectiveness of watermarks. From this work, we hope that future work will continue to challenge and improve existing neural network watermarking strategies as well as explore additional avenues for detecting and removing backdoors in neural networks.

Future work will investigate the feasibility of further decreasing black-box watermark effectiveness in two ways, such as shown in Jia et al.’s recent research (Jia et al., 2020). First,

future work will consider the possibility of boosting a collection of laundered neural networks. Second, future work will consider an adversary's ability to prevent queries to a laundered network when the victim attempts to feed unrelated images for classification. Existing work has already considered detecting out-of-distribution examples (Hendrycks and Gimpel, 2016), and even a poor classifier of this type could further weaken the effectiveness of unrelated watermarks below the acceptable threshold.

To conclude, our results indicate that current work proposing to use backdoors for watermarks in neural networks is overestimating the robustness of these watermarks while underestimating attackers' ability to retain high test accuracy. For the content-based and noise-based watermarks, the watermarks are not robust to detection, reconstruction, and removal attacks. However, all methods do present some additional overhead to model thieves, both in computational resources and the final classification accuracy, particularly in the unrelated style. Our work demonstrates rather that their adoption should not be considered secure against persistent removal attempts as more complex backdoor reconstruction methods are developed.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

CRediT authorship contribution statement

William Aiken: Data curation, Investigation, Methodology, Visualization, Writing - original draft. **Hyoungshick Kim:** Conceptualization, Funding acquisition, Project administration, Supervision, Writing - review & editing. **Simon Woo:** Validation, Writing - review & editing. **Jungwoo Ryoo:** Writing - review & editing.

Acknowledgements

This work was supported by the ICT R&D Programs (no. 2017-0-00545) and the ITRC Support Program (IITP-2019-2015-0-00403). The authors would like to thank all the anonymous reviewers for their valuable feedback.

REFERENCES

- Abiodun OI, Jantan A, Omolara AE, Dada KV, Mohamed NA, Arshad H. State-of-the-art in artificial neural network applications: a survey. *Heliyon* 2018;4(11):e00938.
- Adi Y, Baum C, Cisse M, Pinkas B, Keshet J. Turning your weakness into a strength: watermarking deep neural networks by backdooring. In: 27th USENIX Security Symposium (USENIX Security 18). Baltimore, MD: USENIX Association; 2018. p. 1615–31.
- Atli BG, Szyller S, Juuti M, Marchal S, Asokan N. Extraction of complex DNN models: real threat or boogeyman?. In: *International Workshop on Engineering Dependable and Secure Machine Learning Systems*. Springer; 2020. p. 42–57.
- Chen, X., Liu, C., Li, B., Lu, K., Song, D., 2017. Targeted backdoor attacks on deep learning systems using data poisoning. *arXiv preprint arXiv:1712.05526*
- Cohen, G., Afshar, S., Tapson, J., van Schaik, A., 2017. EMNIST: An extension of MNIST to handwritten letters. *arXiv preprint arXiv:1702.05373*
- Dugas C, Bengio Y, Bélisle F, Nadeau C, Garcia R. Incorporating second-order functional knowledge for better option pricing. In: *Advances in Neural Information Processing Systems*; 2001. p. 472–8.
- Fredrikson M, Jha S, Ristenpart T. Model inversion attacks that exploit confidence information and basic countermeasures. In: *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security*. New York, NY, USA: ACM; 2015. p. 1322–33. doi:10.1145/2810103.2813677.
- Gu, T., Dolan-Gavitt, B., Garg, S., 2017. Badnets: identifying vulnerabilities in the machine learning model supply chain. *arXiv preprint arXiv:1708.06733*
- He K, Zhang X, Ren S, Sun J. Deep residual learning for image recognition. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*; 2016. p. 770–8.
- Hendrycks, D., Gimpel, K., 2016. A baseline for detecting misclassified and out-of-distribution examples in neural networks. *arXiv preprint arXiv:1610.02136*
- Hitaj, D., Mancini, L. V., 2018. Have you stolen my model? Evasion attacks against deep neural network watermarking techniques. *arXiv preprint arXiv:1809.00615*
- Ioffe, S., Szegedy, C., 2015. Batch normalization: accelerating deep network training by reducing internal covariate shift. *arXiv preprint arXiv:1502.03167*
- Jia, H., Choquette-Choo, C. A., Chandrasekaran, V., Papernot, N., 2020. Entangled watermarks as a defense against model extraction. *arXiv preprint arXiv:2002.12200*
- Liu K, Dolan-Gavitt B, Garg S. Fine-pruning: defending against backdooring attacks on deep neural networks. In: *Research in Attacks, Intrusions, and Defenses*; 2018. p. 273–94.
- Liu Y, Ma S, Aafer Y, Lee W-C, Zhai J, Wang W, Zhang X. In: 25nd Annual Network and Distributed System Security Symposium, NDSS 2018, San Diego, California, USA, February 18–22, 2018. Trojaning attack on neural networks. *The Internet Society*; 2018.
- Merrer, E. L., Perez, P., Trédan, G., 2017. Adversarial frontier stitching for remote neural network watermarking. *arXiv preprint arXiv:1711.01894*
- Rouhani B, Chen H, Koushanfar F. In: *Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. Deepsigns: an end-to-end watermarking framework for protecting the ownership of deep neural networks. *ACM*; 2019.
- Shafieinejad, M., Wang, J., Lukas, N., Kerschbaum, F., 2019. On the robustness of the backdoor-based watermarking in deep neural networks. *arXiv preprint arXiv:1906.07745*
- Sun Y, Wang X, Tang X. Deep learning face representation from predicting 10,000 classes. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*; 2014. p. 1891–8.
- Tramèr F, Zhang F, Juels A, Reiter MK, Ristenpart T. Stealing machine learning models via prediction APIs. In: *Proceedings of the 25th USENIX Conference on Security Symposium*. Berkeley, CA, USA: USENIX Association; 2016. p. 601–18.
- Tramèr, F., Papernot, N., Goodfellow, I., Boneh, D., McDaniel, P., 2017. The space of transferable adversarial examples. *arXiv*.
- Uchida Y, Nagai Y, Sakazawa S, Satoh S. Embedding watermarks into deep neural networks. In: *Proceedings of the 2017 ACM on International Conference on Multimedia Retrieval*. New York, NY, USA: ACM; 2017. p. 269–77. doi:10.1145/3078971.3078974.
- Wang B, Yao Y, Shan S, Li H, Viswanath B, Zheng H, Zhao BY. In: 40th IEEE Symposium on Security and Privacy. Neural cleanse: identifying and mitigating backdoor attacks in neural networks. *IEEE*; 2019.

Wolf L, Hassner T, Maoz I. Face Recognition in Unconstrained Videos with Matched Background Similarity. IEEE; 2011.

Zhang J, Gu Z, Jang J, Wu H, Stoecklin MP, Huang H, Molloy I.

Protecting intellectual property of deep neural networks with watermarking. In: Proceedings of the 2018 on Asia Conference on Computer and Communications Security. New York, NY, USA: ACM; 2018. p. 159–72. doi:[10.1145/3196494.3196550](https://doi.org/10.1145/3196494.3196550).



William Aiken (billzo@skku.edu) is a recent Master's student graduate from the Department of Electrical and Computer Engineering at Sungkyunkwan University under the direction of Hyounghick Kim. William is continuing his research into the security of machine learning, neural networks, reinforcement learning, and the Internet of Things at the Altoona campus of the Pennsylvania State University as a research assistant.



Hyounghick Kim (hyoung@skku.edu) is an associate professor in the Department of Computer Science and Engineering, Sungkyunkwan University. He is also working as distinguished visiting researcher at CSIRO Data61. He received a BS degree from the department of Information Engineering at Sungkyunkwan University, a MS degree from the Department of Computer Science at KAIST and a Ph.D. degree from the Computer Laboratory at University of Cambridge in 1999, 2001 and 2012, respectively. After completing his Ph.D., he worked as a post-

doctoral fellow in the Department of Electrical and Computer Engineering at the University of British Columbia. He previously worked for Samsung Electronics as a senior engineer from 2004 to 2008. His current research interest is focused on usable security and security engineering.



Simon S. Woo (swoo@skku.edu) received his M.S. and Ph.D. in Computer Science from Univ. of Southern California (USC), Los Angeles, M.S. in Electrical and Computer Engineering from University of California, San Diego (UCSD), and B.S. in Electrical Engineering from University of Washington (UW), Seattle. He was a member of technical staff (technologist) for 9 years at the NASA's Jet Propulsion Lab (JPL), Pasadena, CA, conducting research in satellite communications, networking and cybersecurity areas. Also, He worked at Intel Corp. and Verisign Research

Lab. Since 2017, he was a tenure-track Assistant Professor at SUNY, South Korea and a Research Assistant Professor at Stony Brook University. Now, he is a tenure-track Assistant Professor at the SKKU Institute for Convergence and Department of Applied Data Science and Software in Sungkyunkwan University, Suwon, Korea.



Jungwoo Ryoo (jryoo@psu.edu) is Professor of Information Sciences and Technology (IST) and the head of the Division of Business, Engineering, and IST at Penn State Altoona. Dr. Ryoo conducts extensive research in software engineering, software security, cybersecurity, security management (in the government sector), software architecture, and requirements engineering. He also has substantial industry experience in architecting and implementing secure, high-performance software for large-scale network management systems. He received his

Ph.D. in computer science from the University of Kansas in 2005.